

visual cheat sheet · beginner → intermediate

Apache Kafka

The complete architecture and data-flow guide — from a single message to a production-grade streaming pipeline.

Producer

Broker

Topic

Partition

Consumer Group

Offset

Replication

Failure Recovery

1



What is Apache Kafka?

The 10-second definition

Apache Kafka is a distributed **event streaming platform**: a system for publishing, storing, and processing continuous streams of records ("events") in real time, at large scale, across many machines.

Producer
writes events



Kafka cluster
stores & orders them



Consumer
reads events

Think of it like...

A postal system where letters (messages) are dropped into labeled mailboxes (topics) and stay there — anyone can walk up and read them, even days later.

Where it came from

Built at LinkedIn to handle huge volumes of activity data, then open-sourced as an Apache project. Now the backbone of real-time data at most large companies.



Key takeaway: Kafka is not just a message queue — it's a **durable, replayable log** that many systems can read independently.



Why Kafka matters

Kafka vs a traditional message queue

Traditional queue	Kafka
Message deleted once read	Message kept for a retention period — replayable
One consumer per message	Many independent consumers can read the same event
Struggles past ~thousands msg/sec	Handles millions of events/sec across partitions
Tight coupling of sender/receiver	Producers and consumers fully decoupled

Real-time analytics

Dashboards updating as events happen

Log aggregation

Collecting logs from many services centrally

Microservice glue

Services talk via events, not direct calls

Event sourcing

Store every state change as an event



Key takeaway: Kafka decouples systems and lets you **replay history** — something a normal queue can't do.

3



Core concepts at a glance

Your reference glossary — bookmark this box

Producer

App that **writes** events to a topic

Broker

One Kafka **server** that stores & serves data

Topic

Named **category** or stream of events

Partition

An **ordered log** — a topic is split into these

Consumer

App that **reads** events from a topic

Consumer group

Consumers that **share** the read workload

Offset

A message's **position number** in a partition

Replication

Copies of a partition on other brokers

4



Topics & partitions

How Kafka organizes data

A **topic** (e.g. `orders`) is split into **partitions** — independent, ordered, append-only logs. This is what makes Kafka scale.

Topic: orders

Partition 0

offsets: 0 · 1 · 2 · 3 · 4 →

Partition 1

offsets: 0 · 1 · 2 →

Partition 2

offsets: 0 · 1 · 2 · 3 →

Order guaranteed

Only **within** a single partition, never across partitions of a topic

Parallelism

More partitions = more consumers can read at once

Example: Topic `orders` has 3 partitions. All events for `order_id=1042` always land in the same partition — so that order's events stay in order.

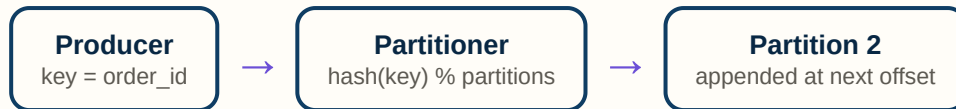
5



Producers — writing data

How an event enters Kafka

A producer sends a record — a **key** + **value** pair — to a topic. Kafka decides **which partition** it lands in.



Same key

Always routes to the **same partition** — preserves per-key order

No key

Spread **round-robin** across partitions for even load

acks=0

Fire and forget — fastest, least safe

acks=1

Leader confirms only

acks=all

All in-sync replicas confirm — safest

6



Brokers & clusters

The storage & serving layer

A **broker** is one Kafka server — it stores partitions and serves reads/writes. A **cluster** is several brokers working together.

Cluster

Broker 1

P0 (leader) · P1

Broker 2

P1 (leader) · P2

Broker 3

P2 (leader) · P0

Controller

One broker manages metadata & leader elections

KRaft mode

Modern Kafka — brokers self-coordinate, **no ZooKeeper** needed

7



Consumers & consumer groups

How data gets read — and shared out

A **consumer group** is a team of consumers sharing the work of reading a topic. Kafka guarantees each partition is read by **only one consumer** in a group at a time.

Topic: orders (3 partitions)

P0

→ Consumer A

P1

→ Consumer B

P2

→ Consumer C

Group: `inventory-service` — 3 partitions, 3 consumers = perfectly balanced

Fewer consumers than partitions

Some consumers handle multiple partitions

More consumers than partitions

Extra consumers sit **idle** — max parallelism = partition count



Key takeaway: Different groups are independent — `inventory-service` and `email-service` can both read the **entire** topic separately (pub/sub fan-out).

8



Offsets — tracking position

Kafka's bookmark system

Every message in a partition gets a sequential **offset** (0, 1, 2, ...). A consumer's offset is its bookmark — how far it has read.

Partition 0 log

[0] [1] [2] [3] [4] [5] [6] [7]

↑ committed offset = 4 ↑ log-end offset = 7 (unread: 5,6,7)

Committed offset

Saved in topic `__consumer_offsets` — survives restarts

Replay

Reset the offset backward to **reprocess** old events

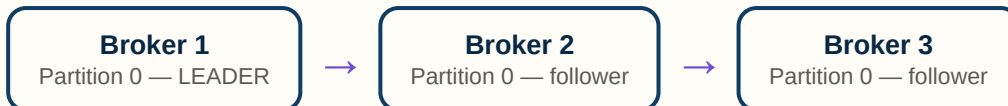
9



Replication — keeping data safe

Copies of every partition, on purpose

Each partition has a **replication factor** — copies (replicas) spread across brokers. One replica is the **leader**; the rest are **followers** that copy from it.



ISR

In-sync replicas — followers fully caught up with the leader

Replication factor 3

Tolerates **2 broker failures** with zero data loss



Key takeaway: Producers & consumers only talk to the **leader**. Followers exist purely for safety.

10



Failure recovery

What happens when a broker dies

Before crash

Broker 1: **leader** P0
Broker 2: follower P0
Broker 3: follower P0

Broker 1 crashes ✗

Controller detects it via heartbeat timeout

After recovery

Broker 2 (was ISR) promoted to **new leader** automatically

Clients refresh their metadata and **reconnect** to the new leader — no manual intervention needed.



Key takeaway: `acks=all` + `min.insync.replicas≥2` means a single broker failure loses **zero** acknowledged data.

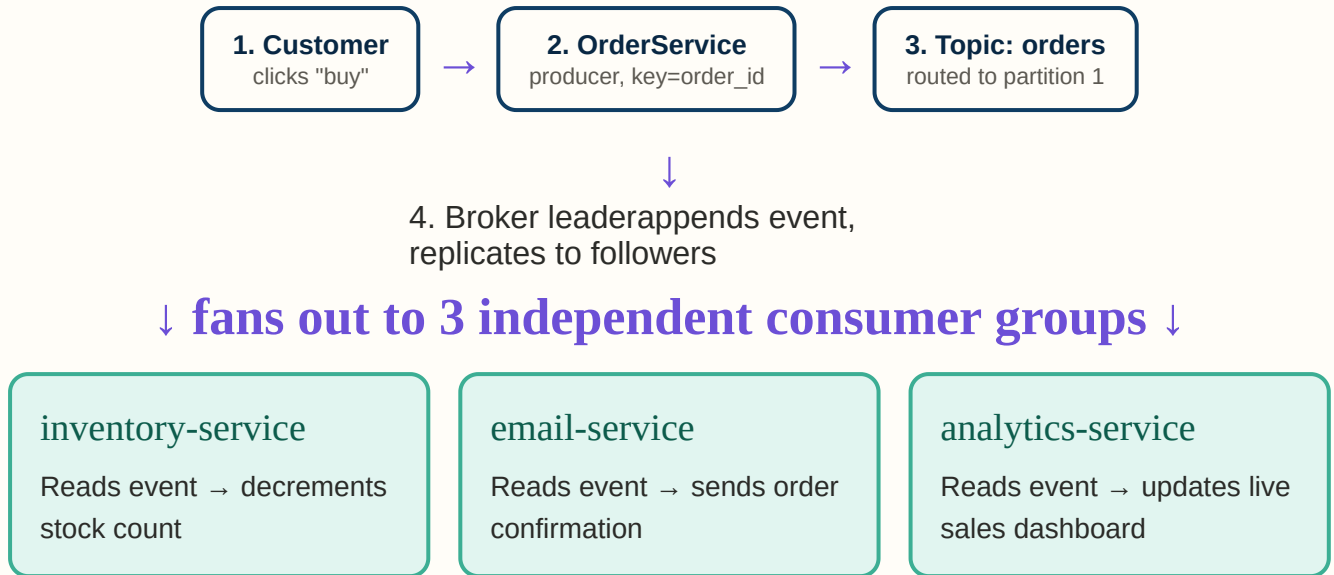
11



Real-world example

End-to-end: an e-commerce order

Every concept above, working together for one customer click:



Notice: Each service reads the **same** `OrderCreated` event independently, at its own pace, from its own committed offset. If `email-service` crashes for 10 minutes, it just resumes from its last offset — no events lost.



Cheat sheet: mistakes, advanced concepts & quick reference

Everything on one page

Common mistakes

- ✗ Too few partitions → **can't scale consumers past partition count**
- ✗ Bad partition key (e.g. constant value) → **creates a "hot" overloaded partition**
- ✗ Replication factor 1 in production → **any broker loss = data loss**
- ✗ Ignoring consumer lag → **monitor it, it signals a struggling consumer**
- ✗ Not handling rebalances → **design consumers to be idempotent**

Advanced concepts

Kafka Streams

Library for processing events in-flight

Schema Registry

Enforces message shape compatibility

Exactly-once

Idempotent producer + transactions

Log compaction

Keeps only the latest value per key

Tiered storage

Offloads old segments to cheap storage

ksqlDB

SQL-like queries over live streams

✓ Golden rule: **Partitions = parallelism. Replication = durability. Consumer groups = scalability.**